

ULPF

Deep-Dive Engineering & Architecture Specification

1. Deconstructing the "Standard 10-Step Pipeline"

Most generic log processing architectures (including the traditional 10-step ASCII pipeline typically proposed) rely on a linear, multi-hop workflow:

```
Ingest → Detect → Vault → Parse → Extract → Custom UEM → Normalize → Validate  
→ Enrich → Sink .
```

While logically sound on paper, **this standard design completely collapses in high-throughput (100k+ EPS), mission-critical government environments** due to four fatal architectural flaws:

- 1. The 7-Hop Latency Trap:** Moving a string through 7 distinct micro-stages (detection, vaulting, regex parsing, extraction, custom mapping, normalization) in Python/Java creates massive CPU context-switching. It limits single-core throughput to ~5,000 EPS.
- 2. The Kernel Socket Bottleneck:** Standard pipelines ingest Syslog via typical TCP/UDP sockets. At 500k EPS (DDoS conditions), the Linux kernel's network stack interrupts will max out the CPU before parsing even begins.
- 3. The "Hash Vault" Illusion:** Standard designs store `hash(event)` in a Raw Vault for immutability. This is mathematically flawed. An attacker can simply `delete` a log from the vault. Since the hashes aren't cryptographically linked, the deletion goes undetected. It proves the log wasn't altered, but **fails to prove the sequence wasn't tampered with**.
- 4. Proprietary UEM (Universal Event Model):** Standard pipelines invent their own "UEM". This creates vendor lock-in and requires custom adapters for every SIEM.

A-HALF (Antigravity High-Assurance Log Fabric) discards this obsolete pipeline in favor of a 2025/2026 state-of-the-art approach utilizing **eBPF, Rust Radix Trees, Merkle Transparency Logs, and Neuro-Symbolic AI**.

2. Deep Dive

Pillar 1: The Data Plane

Goal: Achieve 1.5 Million EPS per core with Zero Data Loss during spikes.

- **eBPF / XDP Zero-Copy Ingestion:** Instead of standard UDP sockets, A-HALF attaches an eBPF (Extended Berkeley Packet Filter) program directly to the NIC driver via XDP (eXpress Data Path). Logs are written directly to Redpanda memory bypassing the Linux kernel network stack.
- **Tiered Redpanda Storage:** Standard queues use fixed RAM buffers (e.g., 128MB drop-oldest), which drop logs during DDoS attacks. We use Redpanda with **Tiered Storage**. As the in-memory ring fills, Redpanda asynchronously flushes cold segments directly to NVMe/S3. This guarantees mathematically **Zero Data Loss** regardless of spike size.
- **O(1) Rust Aho-Corasick Parser:** We collapse the traditional *Detect* → *Parse* → *Extract* → *Normalize* hops into a single pass. The Rust engine builds an **Aho-Corasick Radix Tree** in memory.
 - *Time Complexity:* $O(m)$ where m is the length of the log string.
 - Unlike linear regex lists where parsing slows down as you add more vendors, Radix trees match the log format instantly in a single byte-by-byte traversal, mapping it directly to the **OCSF 1.9 standard** (no proprietary UEM).

Pillar 2: The Integrity Plane

Goal: Cryptographically prove tampering and sequencing, solving the distributed race condition of naive hash-chaining.

- Instead of chaining individual logs (which creates race conditions across distributed queue partitions), A-HALF uses **Batch-based Merkle Trees** (RFC 6962 Standard used in Certificate Transparency).
- **The Math:**
 - Logs are batched into chunks of 10,000 (Parquet format).
 - Leaves: $L_i = \text{SHA256}(\text{Log}_i)$
 - Nodes: $N_{\text{parent}} = \text{SHA256}(N_{\text{left}} + N_{\text{right}})$

- A single **Merkle Root** is generated for the 10k batch and anchored.
- **The Power:** If an auditor wants to verify Log #7,432, we do not need to scan 10,000 hashes. We provide an **Inclusion Proof** in $O(\log n)$ time. This mathematically guarantees the log was untouched, the order is perfect, and scales infinitely across distributed consumers.

Pillar 3: The Security Plane

- Standard systems only alert if parsing failure hits a hard threshold.
- A-HALF asynchronously feeds all **unmapped** fields into a lightweight MiniLM vector embedding model.
- We apply **HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise)** to the stream.
- *Why it wins:* Even if an attacker sends malformed logs at a 2% rate, they will be structurally identical to each other. HDBSCAN detects the *spatial density* of these errors in the vector space and instantly flags it as an Evasion Anomaly, regardless of the overall traffic percentage.

Pillar 4: The AI Plane

Goal: Eliminate LLM Regex Hallucinations that silently corrupt production data.

- Standard "AI log parsers" ask an LLM to generate a Regex. LLMs frequently hallucinate greedy operators (like **.***), which swallow adjacent fields and permanently corrupt Data Lake analytics.
- A-HALF uses **Neuro-Symbolic Program Synthesis** (inspired by Microsoft PROSE).
 1. **Neural Step:** The LLM (DeepSeek) is only used for *Semantic Labeling* (e.g., highlighting which part of the string is an IP vs a Date).
 2. **Symbolic Step:** A deterministic Rust synthesis engine takes these labeled input-output examples and mathematically derives the strictest possible Regex or grok pattern.
- This ensures 100% precision, zero hallucinations, and provably correct parsers.

